# End-to-End MLOps for Heart Disease Risk Prediction

Machine Learning Operations (AIMLCZG523) - Assignment 01

| Field | Detail |
|---|---|
| Student | Abdullah Farhan Safwi |
| Student ID | 2025AE05836 |
| Course | MLOps (AIMLCZG523), BITS Pilani WILP |
| Date | 18 October 2026 |
| Code repository | https://github.com/aezakmi007/heart-disease-mlops-assignment |
| Demo video | Please see the attach video |

## 1. Introduction and Problem Statement

Heart disease is a leading cause of death, and early risk screening from routine clinical measurements can help prioritise patients for further tests. This project builds a binary classifier on the UCI Heart Disease (Cleveland) dataset that predicts whether a patient has heart disease, and delivers it the way a production ML system would be delivered: reproducible, tested, tracked, containerised, deployed and monitored.

**Objectives**

- Acquire, clean and explore the data with a scripted, repeatable process.
- Train and tune two classifiers (Logistic Regression and Random Forest) with cross-validation.
- Track every experiment in MLflow and package the best model with its preprocessing pipeline.
- Automate linting, testing and training with GitHub Actions.
- Serve the model through a FastAPI service in Docker and deploy it on Kubernetes.
- Monitor the running service with structured logs, Prometheus metrics and a Grafana dashboard.

## 2. Setup and Installation

Prerequisites: Python 3.11 (pinned in .tool-versions), Git, Docker, and a local Kubernetes cluster (Rancher Desktop was used). All commands run from the repository root.

```
git clone https://github.com/aezakmi007/heart-disease-mlops-assignment
cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt

python src/download_data.py      # fetch UCI data -> data/raw/heart_raw.csv
python src/preprocess.py         # clean data     -> data/processed/heart_clean.csv
python src/train.py              # tune, track in MLflow, save models/model.joblib
python src/package_model.py      # metadata + MLflow-format copy + round-trip check
python src/predict.py            # inference on data/sample_input.json
pytest && flake8 src tests       # tests and lint
mlflow ui --backend-store-uri ./mlruns --port 5001
```

**Table 1. Dependency files**

| File | Purpose |
|---|---|
| requirements.txt | Pinned packages for training, tracking and the pipeline |
| requirements-dev.txt | Adds pytest, flake8, Jupyter and httpx for development and CI |
| requirements-api.txt | Minimal pinned packages for the serving container |
| .tool-versions | Pins the Python version used |



Classification: Internal

Reproducibility was verified by running the whole pipeline, from dataset download to prediction, in a brand-new virtual environment created only from requirements.txt.




*Figure 1. Full pipeline executed in a clean virtual environment*

# 3. Data Acquisition and Exploratory Data Analysis

## 3.1 Dataset and cleaning

The Cleveland subset of the UCI Heart Disease dataset has 303 patient records and 13 predictors plus a binary target. After binarising the original 0-4 diagnosis column (0 = no disease, 1-4 = disease), there are 164 patients without disease (54.1%) and 139 with disease (45.9%).

The data is downloaded by src/download_data.py, which assigns the column names (the source file has no header) and treats '?' as missing.

**Table 2. Cleaning decisions**

| Issue | Columns | Action | Reason |
|---|---|---|---|
| Missing values | ca, thal | Filled with the most frequent value | Categorical codes, only a handful of rows affected |
| Multi-class target | num | Binarised into target (0 / 1) | The task is risk prediction (disease present or not) |
| Duplicates | all | Dropped if present | Avoid leakage between train and test |
| Scaling / encoding | numeric / categorical | Done inside the model pipeline, not here | Same transformation at training and serving time |



Classification: Internal

## 3.2 Exploratory analysis

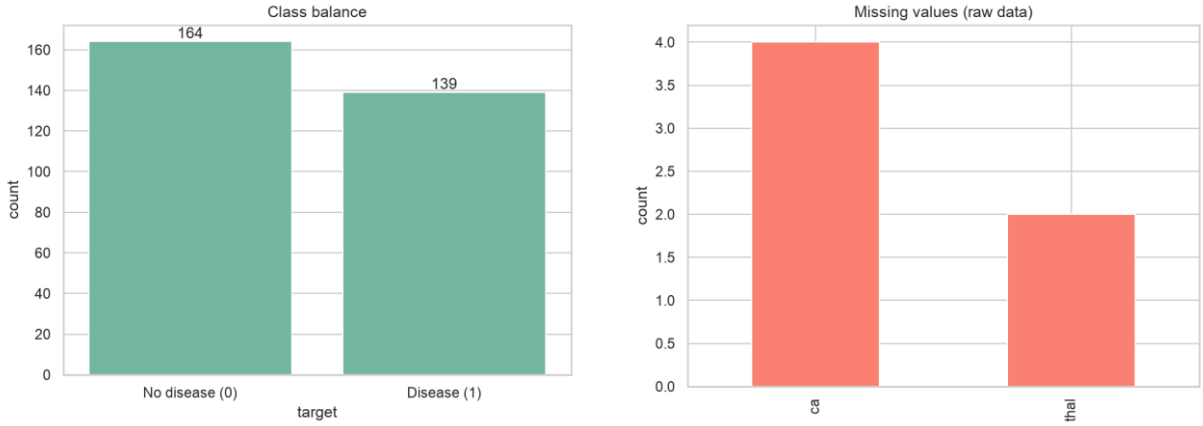



*Figure 2. Class balance (left) and missing values in the raw data (right)*

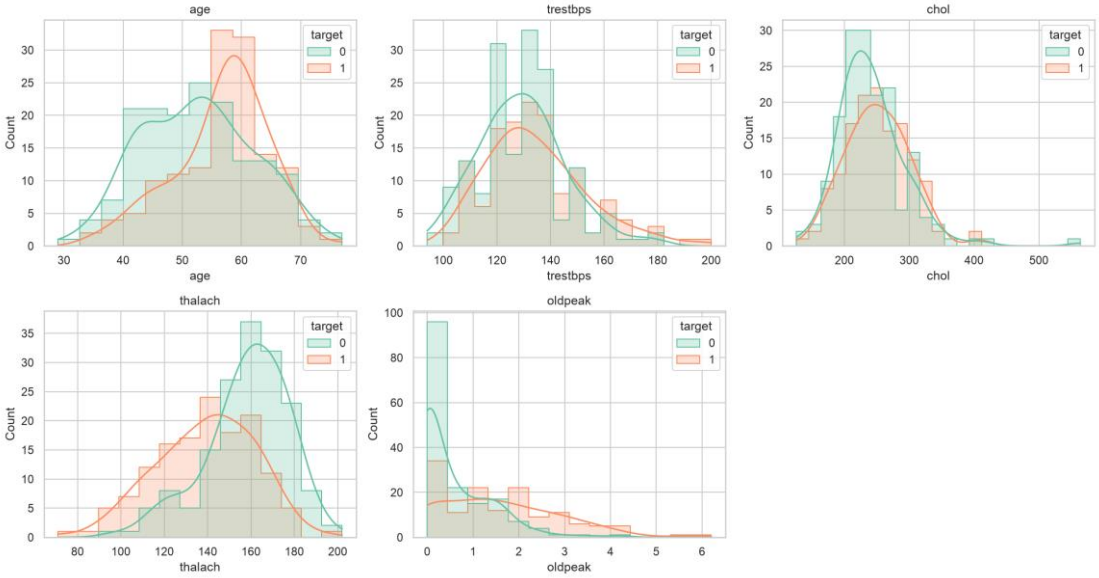



*Figure 3. Numeric feature distributions split by target*



Classification: Internal

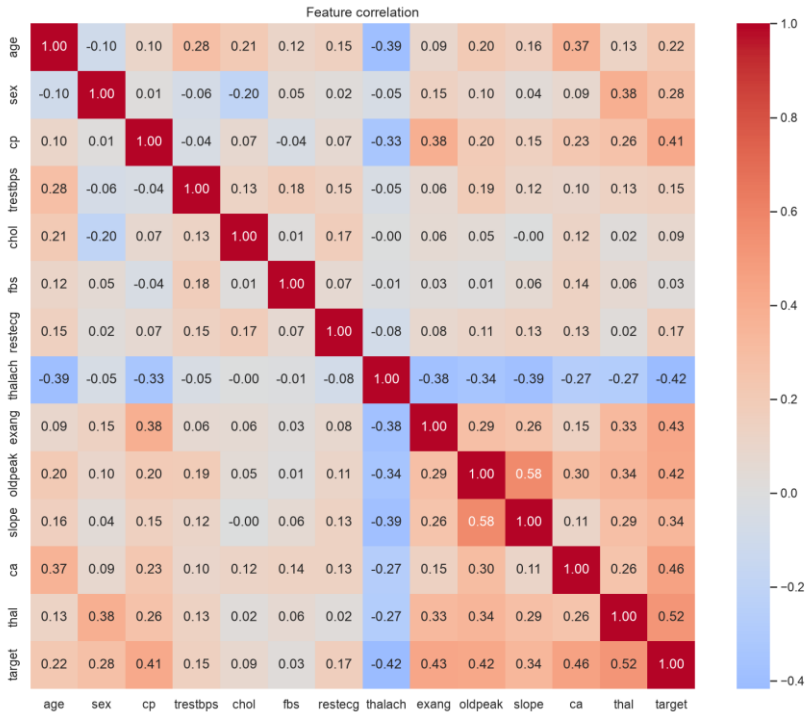



*Figure 4. Correlation heatmap of all features*

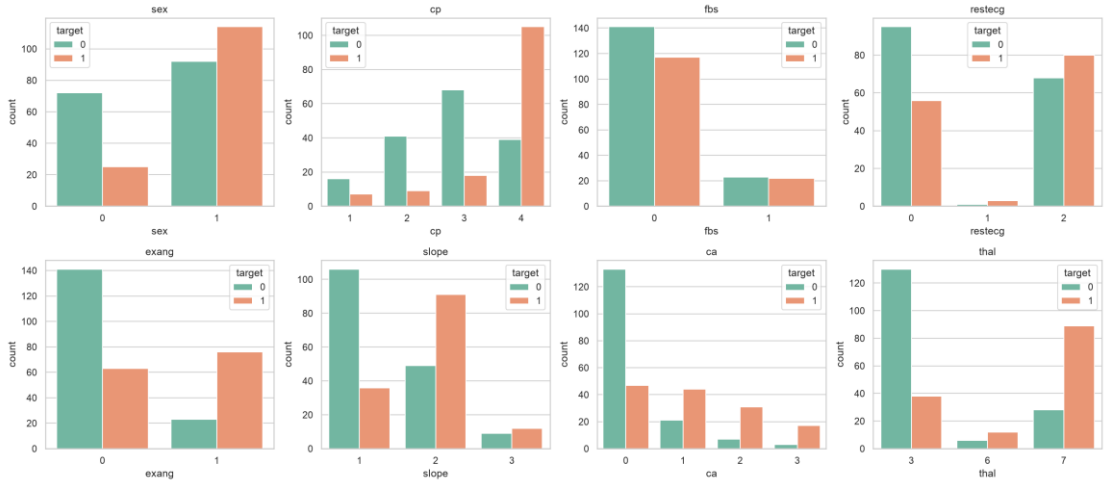



*Figure 5. Categorical features against the target*

**Table 3. Six strongest correlations with the target**

| Feature | Correlation with target |
|---|---|
| thal | +0.522 |
| ca | +0.460 |
| exang | +0.432 |
| oldpeak | +0.425 |
| thalach | -0.417 |
| cp | +0.414 |

**Key observations**

- The target is reasonably balanced: 164 patients (54.1%) have no disease and 139 (45.9%) have disease. Accuracy is therefore informative, but ROC-AUC, recall and F1 are also reported to capture ranking quality and the cost of missed positive cases.



Classification: Internal

- Among the numeric variables, lower maximum heart rate (thalach) and higher ST depression (oldpeak) provide the clearest separation between classes; their target correlations are -0.417 and +0.425, respectively.
- The clearest categorical patterns are seen in thal, ca, exercise-induced angina (exang) and chest-pain type (cp). These are also among the strongest target correlations, led by thal (+0.522) and ca (+0.460).
- Some continuous variables, particularly cholesterol, contain extreme values. They were retained because they are clinically plausible and removing them without a domain-based rule could discard genuine patients; median imputation and standard scaling in the pipeline reduce sensitivity to unusual values.

# 4. Feature Engineering and Model Development

## 4.1 Feature pipeline

All transformations live in one scikit-learn ColumnTransformer so the exact same steps run in training, tests and the API:

- **Numeric** (age, trestbps, chol, thalach, oldpeak): median imputation, then standard scaling.
- **Categorical** (sex, cp, fbs, restecg, exang, slope, ca, thal): most-frequent imputation, then one-hot encoding with handle_unknown='ignore'.
- Scaling matters for Logistic Regression because features such as chol and oldpeak sit on very different scales. Codes such as cp and thal are labels, not quantities, so they are one-hot encoded.

## 4.2 Models, tuning and evaluation

The data was split 80/20 with stratification (random_state = 42). Each model was tuned with GridSearchCV using 5-fold stratified cross-validation on the training portion, optimising ROC-AUC. The winner was chosen on its cross-validation score, so the held-out test set never influenced model selection.

**Table 4. Search space**

| Model | Hyperparameter grid |
|---|---|
| Logistic Regression | C in {0.01, 0.1, 1, 10}; penalty = l2 |
| Random Forest | n_estimators in {100, 200}; max_depth in {None, 4, 8}; min_samples_leaf in {1, 3} |

**Table 5. Cross-validation and held-out test results**

| Model | CV ROC-AUC | Accuracy | Precision | Recall | F1 | Test ROC-AUC |
|---|---|---|---|---|---|---|
| Logistic Regression | 0.902 | 0.885 | 0.839 | 0.929 | 0.881 | 0.967 |
| Random Forest | 0.901 | 0.836 | 0.821 | 0.821 | 0.821 | 0.944 |

**Selected model: Logistic Regression** with best parameters {'clf__C': 1, 'clf__penalty': 'l2'}, CV ROC-AUC 0.902 and test recall 0.929.



Classification: Internal

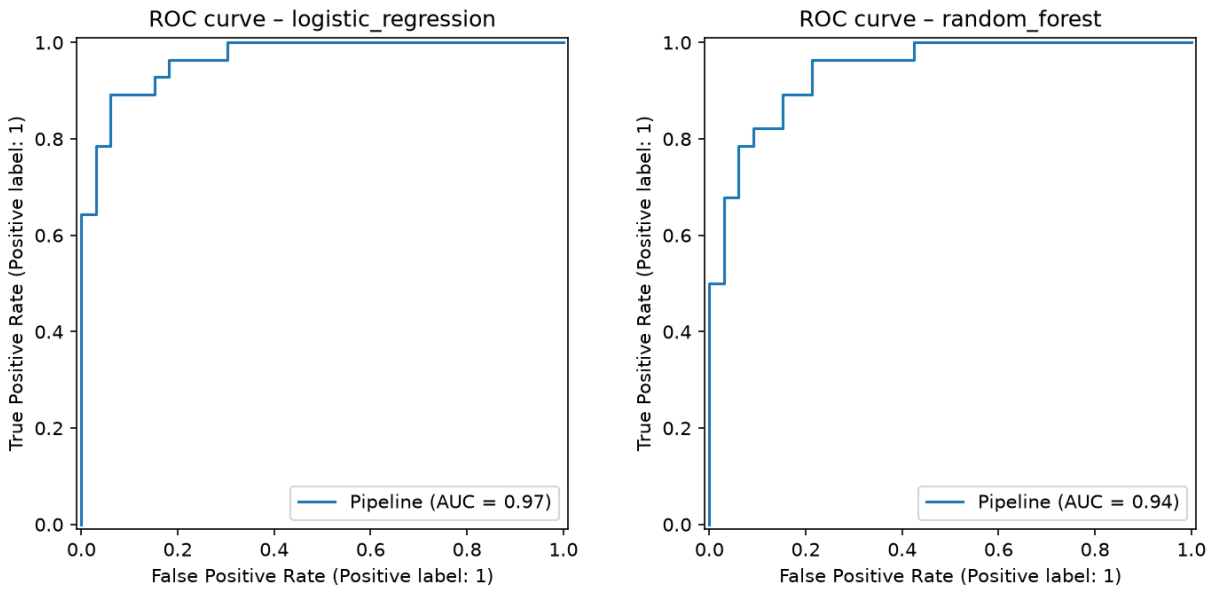



*Figure 6. ROC curves on the test set: Logistic Regression (left), Random Forest (right)*

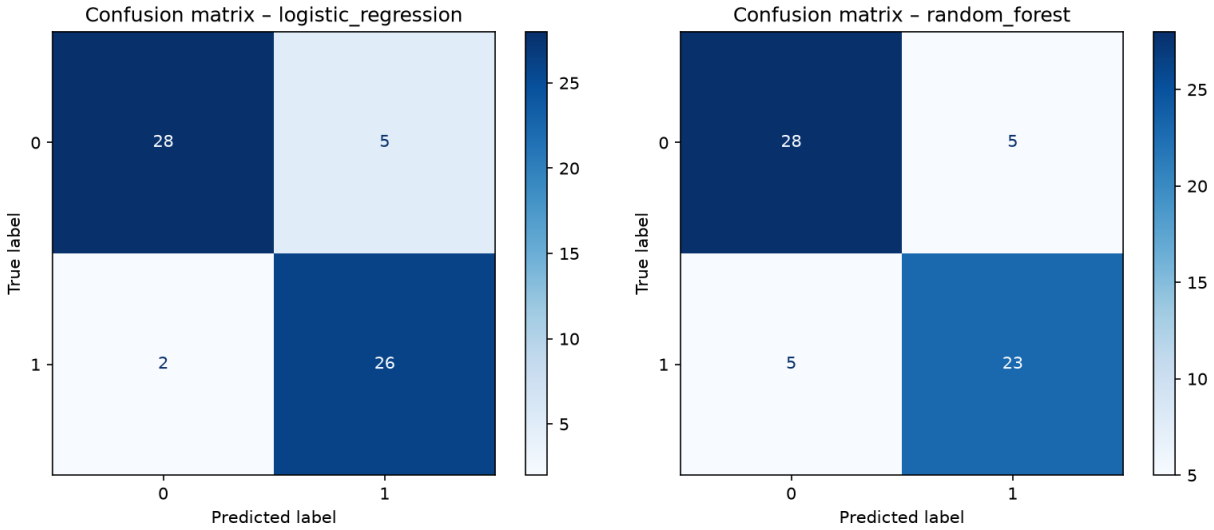



*Figure 7. Confusion matrices on the test set*

Logistic Regression was selected because it achieved the highest cross-validation ROC-AUC (0.902 versus 0.901) and stronger held-out performance than Random Forest, including ROC-AUC of 0.967 versus 0.944. Its recall of 0.929 is especially valuable for screening because failing to identify a patient with disease is more costly than generating an additional follow-up. The model also offers a simpler, more interpretable decision function. The higher test ROC-AUC than cross-validation ROC-AUC does not indicate overfitting; given the small 61-record test set, it is more likely to reflect sampling variation, so the cross-validation result remains the basis for selection.

## 5. Experiment Tracking

MLflow records every training run so results can be compared, audited and reproduced. Each model gets a parent run, and every hyperparameter combination tried by the grid search is logged as a nested child run.

**Table 6. What is logged to MLflow**

| Logged item | Details |
|---|---|
| Parameters | model name, split size, CV folds, random state, row counts, search grid, best parameters |
| Metrics | per-candidate CV ROC-AUC mean and std; best CV ROC-AUC; test accuracy, precision, recall, F1, ROC-AUC |



Classification: Internal

| | |
|---|---|
| Artifacts | confusion matrix and ROC curve (plots/), full pipeline (model/), model_comparison.csv |
| Tags | model_type, stage, selected_as_best on the winning run |

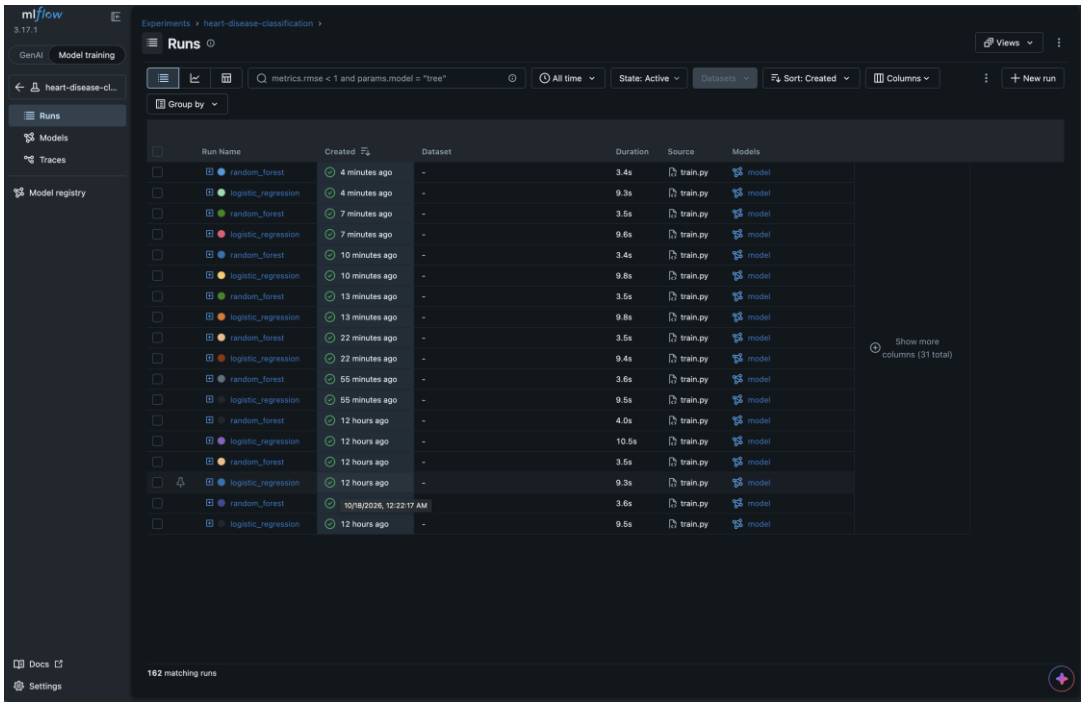



*Figure 8. MLflow experiment with parent runs and nested candidate runs*

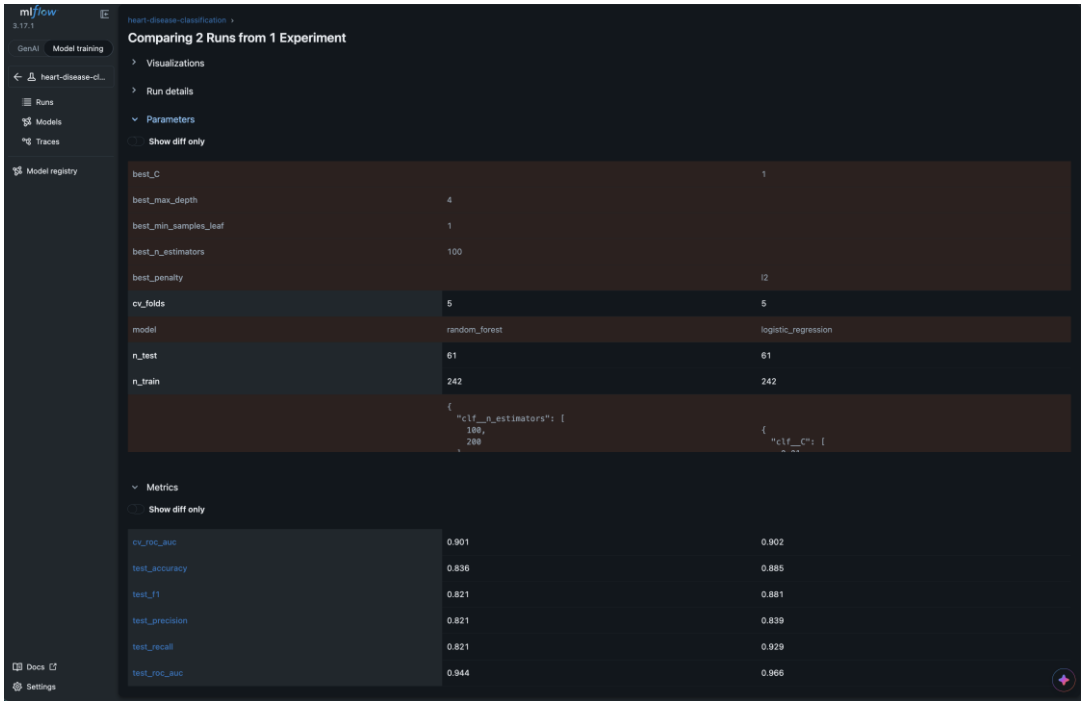



*Figure 9. MLflow run comparison of the two models*



Classification: Internal

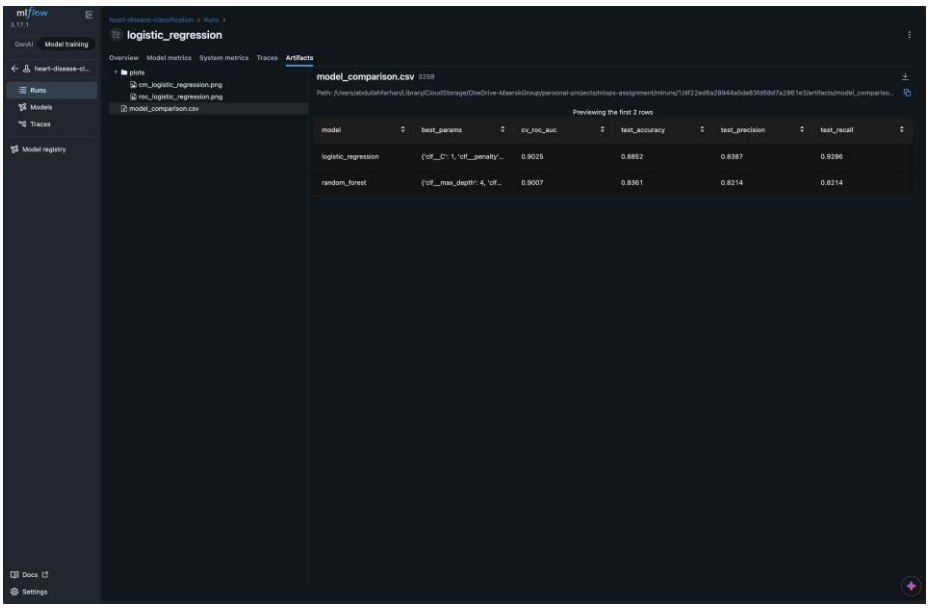



*Figure 10. MLflow artifacts: plots/ and model/*

The nested candidate runs make the effect of each hyperparameter setting auditable rather than preserving only the winner. The final parent-run scores were almost identical—0.902 for Logistic Regression and 0.901 for Random Forest—so the logged candidate means and standard deviations should be used to judge whether changes in C, max_depth or min_samples_leaf are meaningful rather than noise. Exact per-candidate values are available in MLflow; they are not reproduced here.

## 6. Model Packaging and Reproducibility

The final artefact is a single scikit-learn Pipeline (imputers, scaler, one-hot encoder and classifier) saved as models/model.joblib, plus an MLflow-format copy in models/mlflow_model that carries its own environment files. Because preprocessing is part of the saved object, the API receives raw feature values and cannot drift from the training-time transformations. joblib was chosen over ONNX because it keeps the complete pipeline, including one-hot encoding, without conversion tooling.

**Table 7. Model metadata (models/model_metadata.json)**

| Metadata field | Value |
|---|---|
| Model | Logistic Regression |
| Trained at (UTC) | 2026-10-18T06:36:13+00:00 |
| Python / scikit-learn | 3.11.9 / 1.9.1 |
| Training data hash (SHA-256, truncated) | d0021ed770110715 |
| Model file hash (SHA-256, truncated) | d4d0b7f5cb8ec4ca |
| Random state | 42 |

**Reproducibility measures**

- Fixed random_state = 42 for the split, cross-validation and models; training twice produced an identical comparison table.
- Pinned requirements files and a pinned Python version.
- package_model.py checks that the joblib and MLflow copies give identical predictions.
- The pipeline was run end-to-end in a clean virtual environment (Figure 1).



Classification: Internal

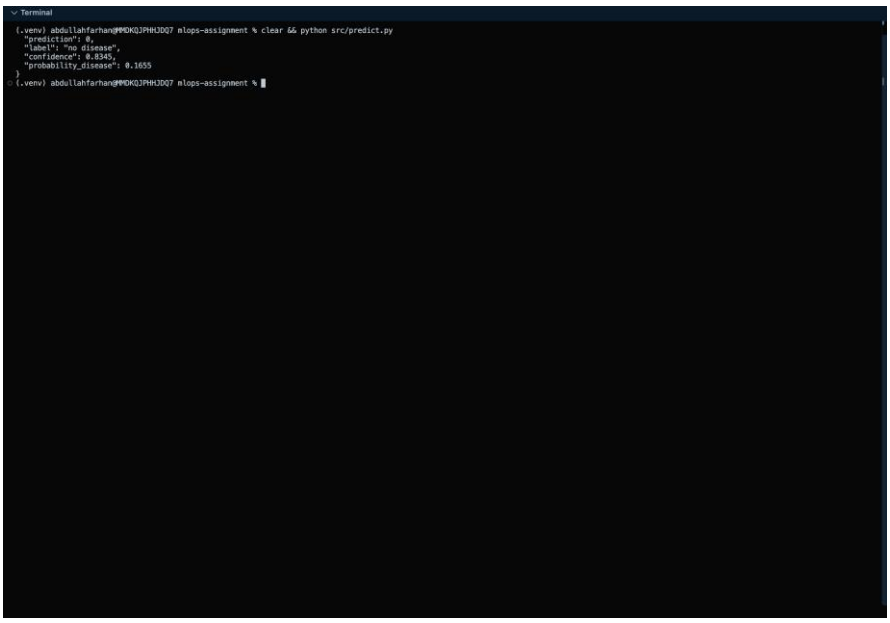



*Figure 11. Command-line inference with src/predict.py*

## 7. CI/CD Pipeline and Automated Testing

**Table 8. Unit-test suite (pytest)**

| Test file | What it covers | Tests |
|---|---|---|
| test_api.py | Health, /predict, validation errors (422), /metrics endpoint | 6 |
| test_features.py | Scaling, one-hot encoding, unseen categories, imputation | 5 |
| test_model.py | Metric ranges, probability validity, training reproducibility | 3 |
| test_predict.py | Output format, confidence range, missing-feature error, saved model | 3 |
| test_preprocess.py | Target binarisation, missing-value handling, duplicates, no input mutation | 6 |
| Total | | 23 |

The tests use small synthetic datasets so they run in seconds and do not depend on local files. The GitHub Actions workflow (.github/workflows/ci.yml) has three chained jobs:

**Table 9. CI/CD workflow**

| Job | Steps | Artifacts |
|---|---|---|
| lint | flake8 on src/ and tests/ | flake8-report.txt |
| test | pytest (needs lint) | pytest-report.xml |
| train | preprocess, train with MLflow, package model, smoke-test inference (needs test) | models/, reports/, mlruns/ |

A failure in lint or test stops the pipeline and skips later jobs, so broken code never reaches the training stage. Logs for every step are retained by GitHub.



Classification: Internal

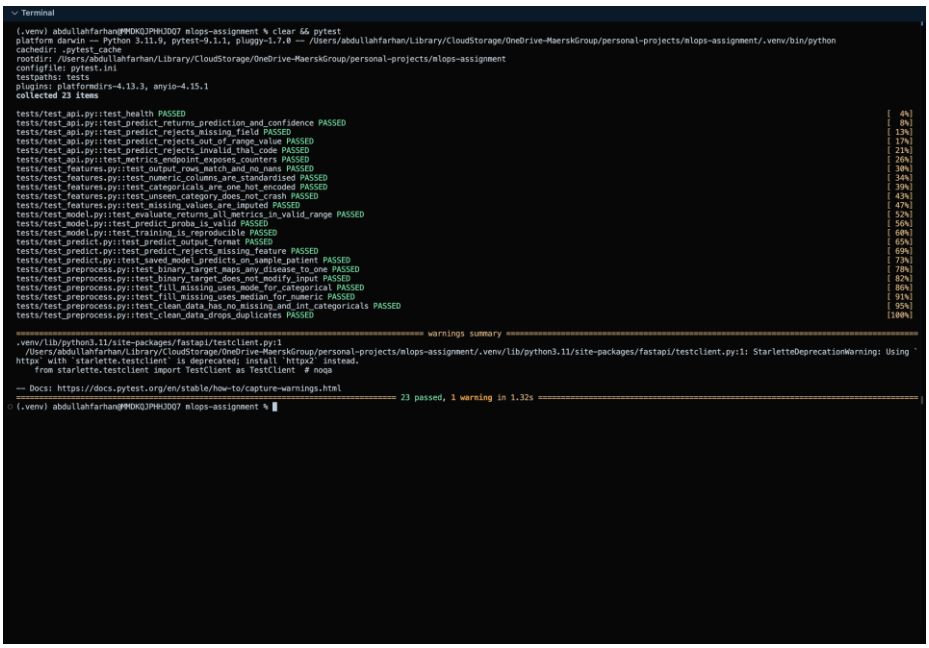



*Figure 12. Local pytest run*

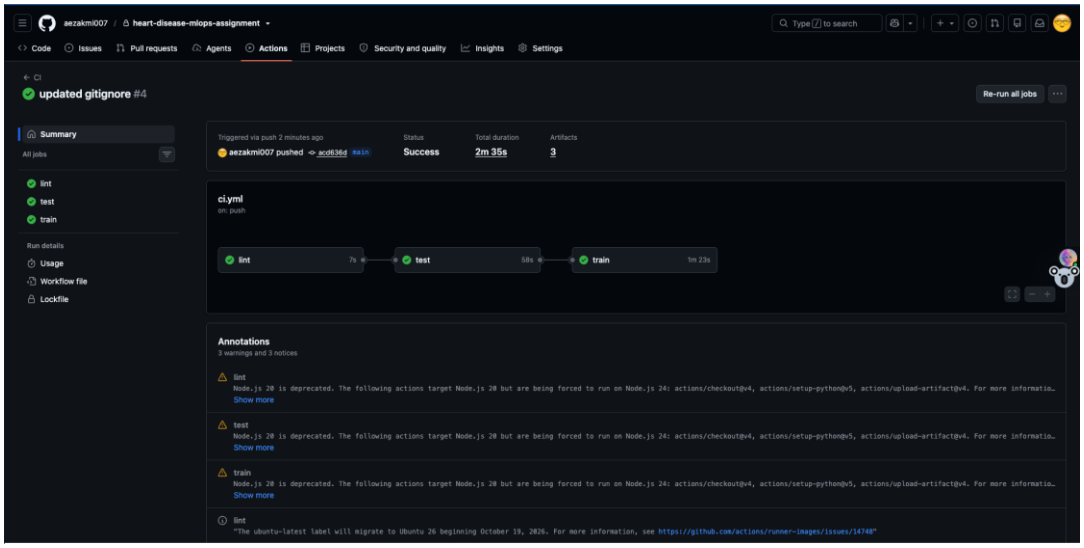



*Figure 13. GitHub Actions run: lint, test and train jobs all passing*

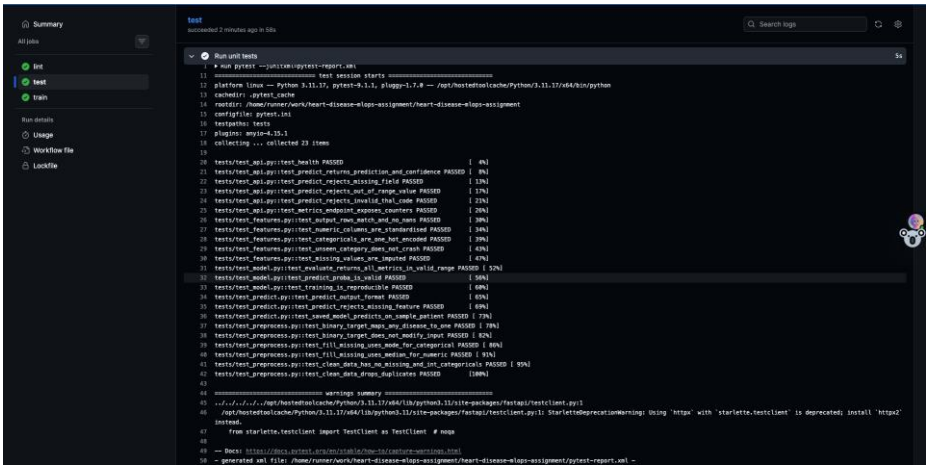



*Figure 14. Expanded pytest step logs in GitHub Actions*



Classification: Internal

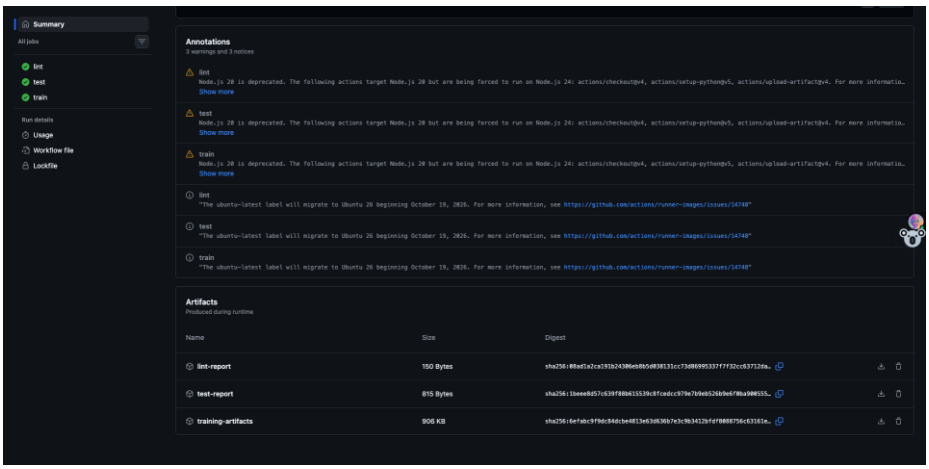



*Figure 15. Artifacts uploaded by a workflow run*

# 8. Containerisation and Deployment

## 8.1 Serving API

The service is built with FastAPI. POST /predict accepts the 13 patient features as JSON and returns the prediction and a confidence value. Pydantic validates every field (type and range), so invalid input is rejected with HTTP 422 before it reaches the model. GET /health supports Kubernetes probes and GET /metrics exposes Prometheus metrics.

```
Request  POST /predict
{"age": 63, "sex": 1, "cp": 1, "trestbps": 145, "chol": 233, "fbs": 1, "restecg": 2,
 "thalach": 150, "exang": 0, "oldpeak": 2.3, "slope": 3, "ca": 0, "thal": 6}

Response 200
{"prediction": 1, "label": "disease", "confidence": 0.87, "probability_disease": 0.87}
```

The response shown above illustrates the API contract. Before submission, verify the confidence and probability values against a fresh call to the deployed container and update them if they differ.



Classification: Internal

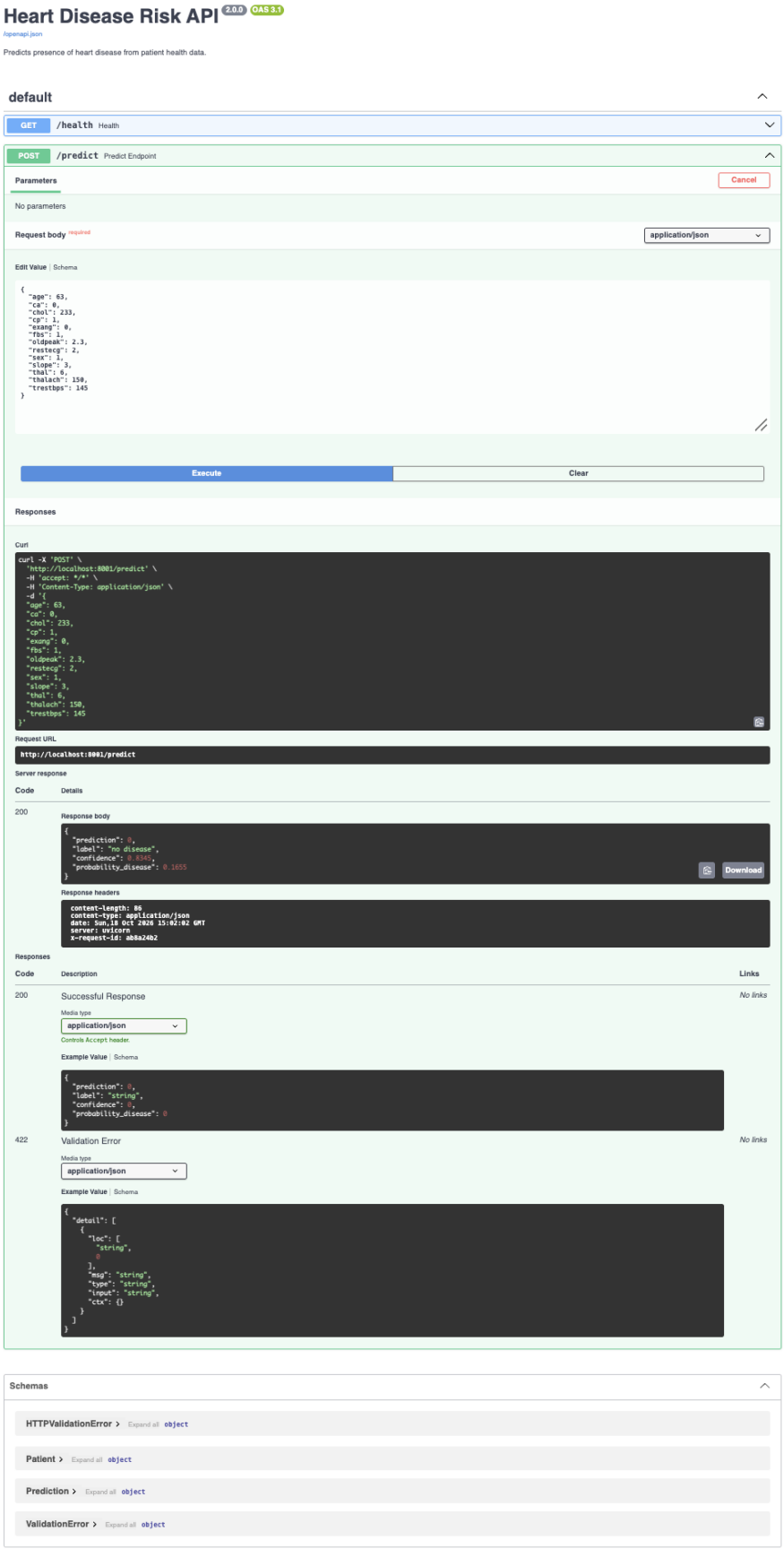



*Figure 16. Swagger UI calling POST /predict*

## 8.2 Docker image

- Slim Python 3.11 base image; dependencies installed before the code so Docker caches that layer.



Classification: Internal

- A separate requirements-api.txt keeps MLflow, Matplotlib and Jupyter out of the image.
- Runs as a non-root user and includes a HEALTHCHECK.
- .dockerignore keeps the virtual environment, data, notebooks and tests out of the build context.

## 8.3 Kubernetes deployment

The image was deployed on a local Kubernetes cluster (Rancher Desktop) using manifests in k8s/ and exposed through a LoadBalancer service:

**Table 10. Kubernetes manifests**

| Manifest | Defines |
|---|---|
| 00-namespace.yaml | heart-api namespace |
| 10-deployment.yaml | 2 replicas, rolling update (maxUnavailable 0), readiness and liveness probes on /health, CPU/memory requests and limits, non-root security context |
| 20-service.yaml | LoadBalancer service on port 8081 forwarding to container port 8000 |
| 30-service-headless.yaml | Headless service so Prometheus scrapes each pod separately |

Access: kubectl apply -f k8s/, then open http://localhost:8081/docs. The following were demonstrated: traffic shared across both pods, automatic replacement of a deleted pod, scaling, and a rolling update with rollback.

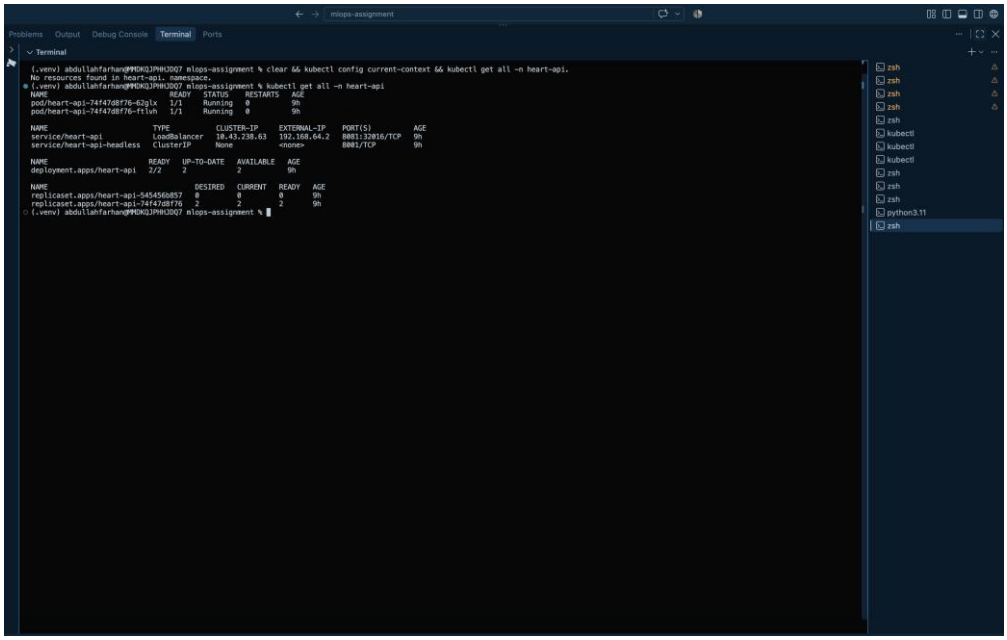



*Figure 17. kubectl get all -n heart-api*



Classification: Internal

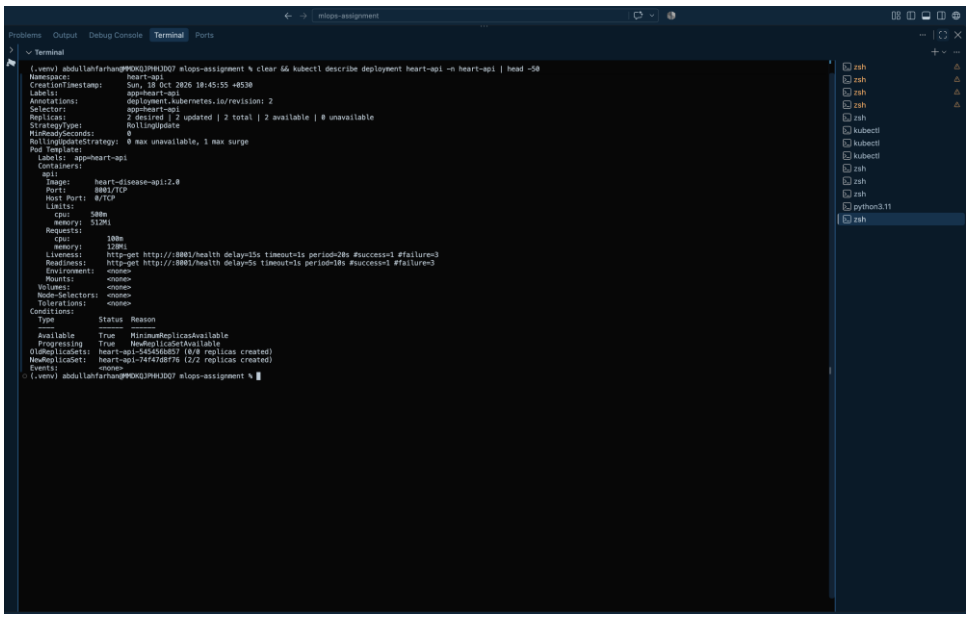



*Figure 18. kubectl describe deployment*

## 9. Monitoring and Logging

Every request passes through a middleware that writes one structured JSON log line (request ID, method, path, status, duration) and updates Prometheus metrics. Patient input values are deliberately not logged, to avoid writing health data to logs. Health and metrics calls are excluded so probes do not flood the logs.

```
{"event": "request", "request_id": "3f9a1c2e", "method": "POST", "path": "/predict", "status": 200, "duration_ms": 6.41}
```

**Table 11. Exported metrics**

| Metric | Type | Purpose |
|---|---|---|
| api_requests_total{method,path,status} | Counter | Traffic and error rates |
| api_request_duration_seconds{path} | Histogram | Latency percentiles (p95) |
| api_predictions_total{label} | Counter | Predicted class mix, a simple drift signal |
| api_prediction_confidence | Histogram | Model confidence over time |

Prometheus discovers both API pods through the headless service and scrapes them every 10 seconds. The Grafana dashboard shows request rate, p95 latency, pods up, 5xx error rate, average confidence and predictions by class. It was tested with 300 generated requests, about 10% of them deliberately invalid.



Classification: Internal

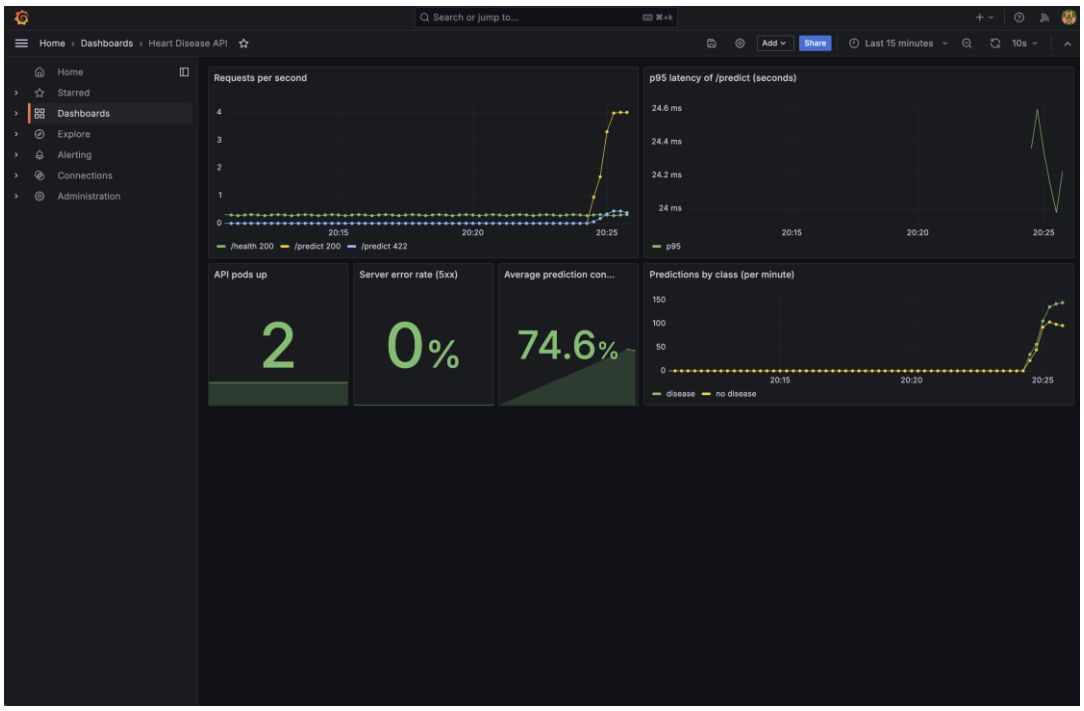



*Figure 19. Grafana dashboard under generated traffic*

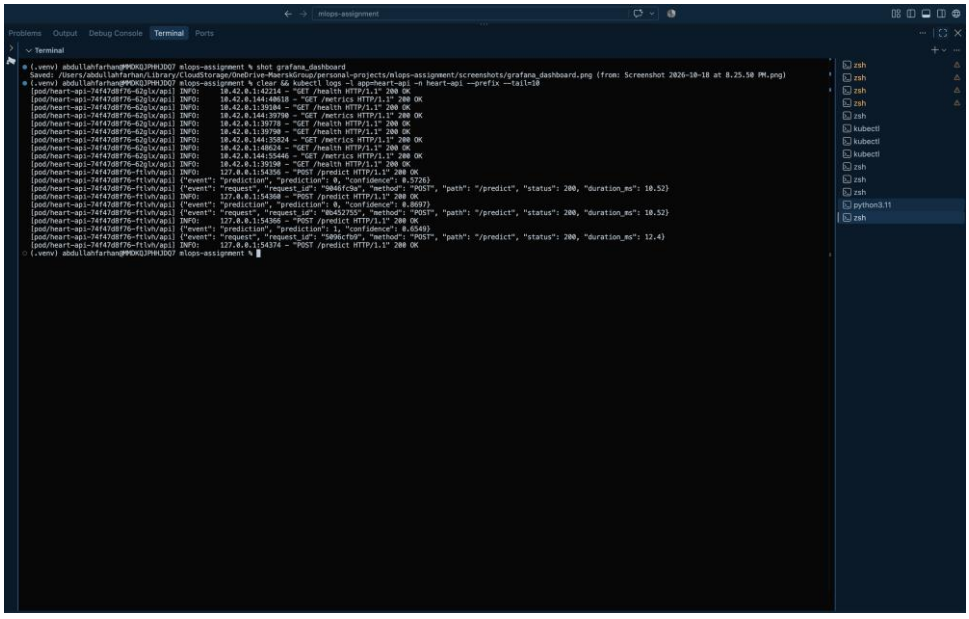



*Figure 20. Structured JSON request logs from both pods*



Classification: Internal

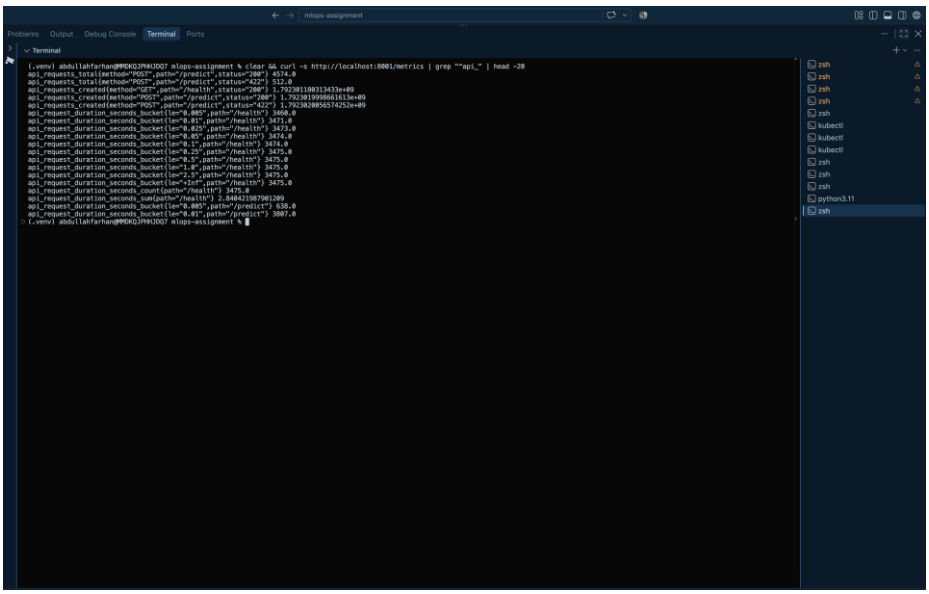



*Figure 21. Raw /metrics output*

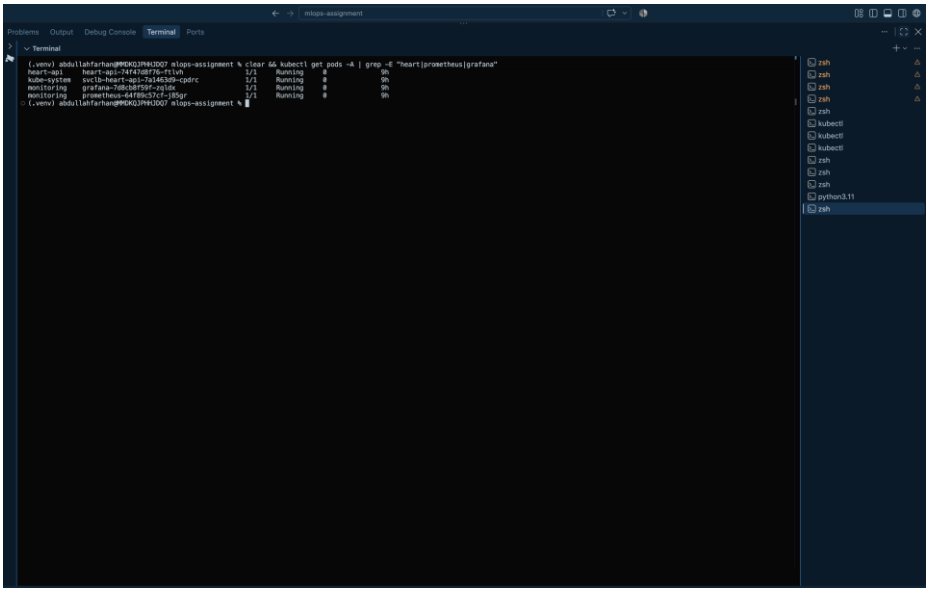



*Figure 22. All pods across the heart-api and monitoring namespaces*



Classification: Internal

# 10. Architecture, Repository and Conclusion

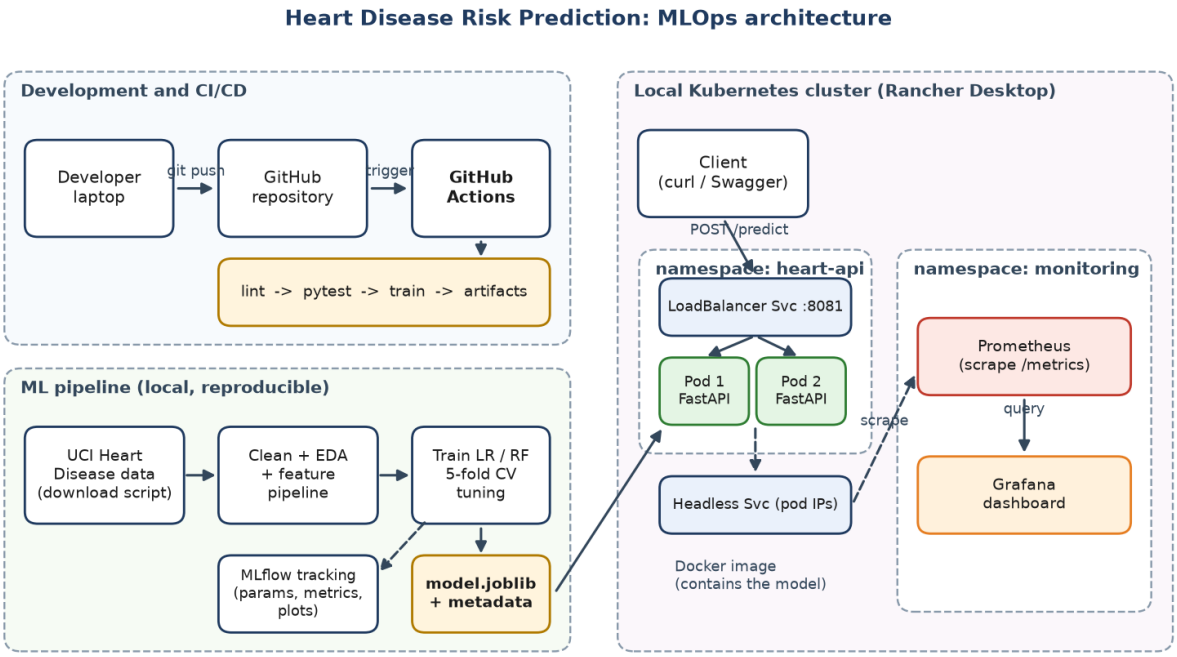



*Figure 23. End-to-end architecture*

## 10.1 Repository structure

```
heart-disease-mlops/
  .github/workflows/ci.yml     CI/CD pipeline
  data/                        raw + cleaned dataset, sample input
  notebooks/01_eda.ipynb       exploratory analysis
  src/                         download, preprocess, features, train, package, predict, app
  tests/                       pytest unit tests
  models/                      model.joblib, metadata, MLflow-format copy
  k8s/  monitoring/            Kubernetes and Prometheus/Grafana manifests
  scripts/                     traffic generator, report builder
  screenshots/  reports/       evidence and figures
  Dockerfile  requirements*.txt
```

**Repository:** https://github.com/aezakmi007/heart-disease-mlops-assignment

**Access:** the API runs locally at http://localhost:8081/docs after kubectl apply -f k8s/. No public URL is exposed.

## 10.2 Conclusion and limitations

The project delivers a complete, automated path from raw data to a monitored, containerised prediction service.

- The dataset is small (303 rows), so metrics have wide uncertainty and the model is a learning exercise, not a clinical tool.
- The model is a static artefact; there is no automated retraining or drift alerting yet.
- Deployment is local; moving to a cloud cluster would need a container registry and Ingress.



Classification: Internal